

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
по дисциплине
SDR

по теме:

Имитация аналоговой передачи звука и его прием с использованием SDR. Анализ влияния чувствительности приемника и усиления передатчика на качество принятых отсчетов сигнала (семплов)

Студент:
Группа ИА-331

К.А Любимов

Предподаватели:
Лектор
Практик
Практик

Калачиков А.А
Ахпашев А.В
Попович И.А

Новосибирск 2025 г.

1 ИМИТАЦИЯ АНАЛОГОВОЙ ПЕРЕДАЧИ ЗВУКА И ЕГО ПРИЕМ С ИСПОЛЬЗОВАНИЕМ SDR. АНАЛИЗ ВЛИЯНИЯ ЧУВСТВИТЕЛЬНОСТИ ПРИЕМНИКА И УСИЛЕНИЯ ПЕРЕДАТЧИКА НА КАЧЕСТВО ПРИНЯТЫХ ОТСЧЕТОВ СИГНАЛА (СЕМПЛОВ)

[Ссылка на GitHub](#)

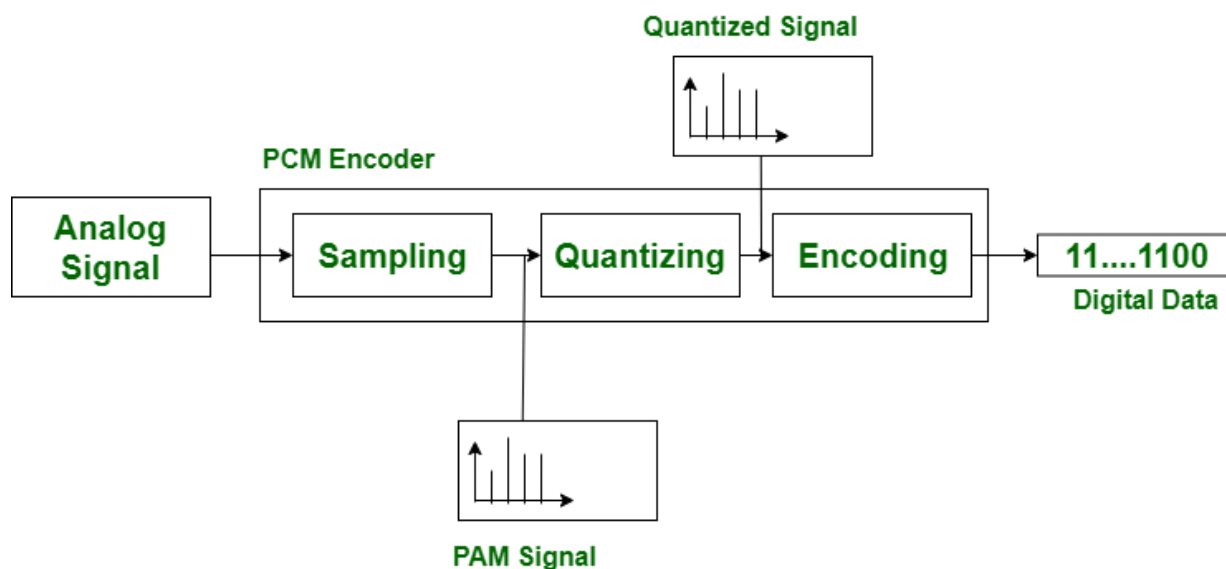
Цель практики:

Попробовать отправить свою аудио запись, получить ее и посмотреть что поменялось. Попробовать получить эффект сложения сигналов, путем скрещения антенн двух Pluto SDR.

Краткие теоретические сведения

Сперва что такое MP3, PCM?

PCM (Pulse Code Modulation) — это линейное, несжатое представление звука. Сигнал записывается как последовательность отсчётов амплитуды с фиксированной частотой дискретизации и глубиной квантования. PCM хранит аудио без сжатия, что делает его максимально точным, но объёмным.



Block Diagram Of PCM

Рис. 1 — Графики

MP3 (MPEG-1 Audio Layer III) — это формат сжатия аудио с потерями, основанный на психоакустической модели. Он удаляет компоненты звука, менее заметные человеческому слуху, и применяет дополнительные методы кодирования, что существенно уменьшает размер файла.

Для конвертации между этими форматами можно использовать библиотеку `pydub` в Python. А для конвертации MP3 в WAV можно использовать `ffmpeg`.

Листинг 1.1 — Код на C++ для чтения PCM файла

```

1 int16_t *read_pcm(const char *filename, size_t *sample_count)
2 {
3     FILE *file = fopen(filename, "rb");
4
5     fseek(file, 0, SEEK_END);
6     long file_size = ftell(file);
7     fseek(file, 0, SEEK_SET);
8     printf("file_size = %ld\n", file_size);
9
10    *sample_count = file_size / sizeof(int16_t);
11    int16_t *samples = (int16_t *)malloc(file_size);
12    size_t sf = fread(samples, sizeof(int16_t), *sample_count, file);
13
14    if (sf == 0){
15        printf("file %s empty!", filename);
16    }
17
18    fclose(file);
19
20    return samples;
21 }

```

Выполнение

Для начала, я написал код для конвертации mp3 файла в pcm формат с помощью pydub.

Листинг 1.2 — Код на python для конвертации mp3 в pcm

```

1 import numpy as np
2 import librosa
3 from pydub import AudioSegment
4
5 def mp3_to_pcm(src_path, dst_path, sample_rate=44100):
6     audio_data, sr = librosa.load(src_path, sr=sample_rate, mono=False)
7     if audio_data.ndim > 1:
8         audio_data = audio_data.T.flatten()
9     pcm_array = np.int16(audio_data * 32767)
10    pcm_array.tofile(dst_path)
11
12 def pcm_to_mp3(pcm_path, output_path, channels=2, sample_rate=44100, bitrate="160k"):
13     raw_audio = np.fromfile(pcm_path, dtype=np.int16)
14     sound = AudioSegment(
15         data=raw_audio.tobytes(),
16         sample_width=2,
17         frame_rate=sample_rate,
18         channels=channels
19     )
20     sound.export(output_path, format="mp3", bitrate=bitrate)
21
22 print("""PCM->MP3: 0
23 MP3->PCM: 1""")
24 ans = int(input("Выберите режим: "))
25 if ans == 1:
26     mp3_to_pcm("../1.mp3", "../1.pcm")
27 else:
28     pcm_to_mp3("../2.pcm", "../2.mp3")

```

Он на выбор преобразует mp3 файл в pcm или наоборот. Используем его перед началом экспериментов. В main.cpp я добавил чтение pcm файла и заполнение буфера прочитанными значениями.

Листинг 1.3 — Заполнения буфера данными из pcm

```
1 size_t iteration_count = 10;
2 long long last_time = 0;
3
4 FILE* file = fopen("../rx.pcm", "wb");
5 FILE* file1 = fopen("../tx.pcm", "wb");
6
7 for (size_t buffers_read = 0; buffers_read < iteration_count; buffers_read++)
8 {
9     void *rx_buffs[] = {rx_buffer};
10    int flags;           // flags set by receive operation
11    long long timeNs;    //timestamp for receive buffer
12    long timeoutUs = 1000000;
13    int sr = SoapySDRDevice_readStream(sdr, rxStream, rx_buffs, rx_mtu, &flags, &
14        timeNs, timeoutUs);
15
16    printf("Buffer: %lu - Samples: %i, Flags: %i, Time: %lli, TimeDiff: %lli\n",
17        buffers_read, sr, flags, timeNs, timeNs - last_time);
18    last_time = timeNs;
19
20    long long tx_time = timeNs + (4 * 1000 * 1000); // на 4 мс[] вбудущее
21
22    for(size_t i = 0; i < 8; i++)
23    {
24        uint8_t tx_time_byte = (tx_time >> (i * 8)) & 0xff;
25        tx_buff[2 + i] = tx_time_byte << 4;
26    }
27
28    void *tx_buffs[] = {tx_buff};
29    if( (buffers_read==2) ){
30        flags = SOAPY_SDR_HAS_TIME;
31        int st = SoapySDRDevice_writeStream(sdr, txStream, (const void * const*)
32            tx_buffs, tx_mtu, &flags, tx_time, timeoutUs);
33        if ((size_t)st != tx_mtu)
34        {
35            printf("TX Failed: %i\n", st);
36        }
37    }
38
39    fwrite(rx_buffer, 2 * rx_mtu * sizeof(int16_t), 1, file);
40    fwrite(tx_buffs, 2 * rx_mtu * sizeof(int16_t), 1, file1);
41
42    }
43
44    fclose(file);
```

Не забываем закрыть файл через fclose, и получается, что мы записали и отправили наш pcm файл через Pluto SDR, rx_buffer в file(rx.pcm) и tx_buffer в file1(tx.pcm). С помощью нашего кода на python мы можем конвертировать pcm обратно в mp3, чтобы прослушать, полученные mp3 получились искаженными. Если скрестить две Pluto SDR, одновременно начать отправку сэплов, то получится эффект наложения сигналов. У нас с одnogруппником получилось так, что одновременно были слышны 2 песни.

Вывод

Я научился переводить файлы из mp3 в pcm и обратно, записывать их в буфер Pluto SDR, отправлять их и получать, полученные файлы были конвертированы обратно в mp3 для того чтобы прослушать, что получилось передать.